Algorithms

BELLMAN FORD & FLOYD WARSHALL

Last Week

- Shortest paths (SP):
  - Relax
  - SP in DAGs
  - Dijkstra's algorithm

This Week

- Shortest paths (SP):
  - Bellman Ford
  - Floyd Warshall

Bellman Ford

- Single source s
- Supports graphs with arbitrary weights
- Can be used to find negative cycles

Algorithm 1 INIT

```
1: for each vertex v ∈ G.V do
2:     v.d = ∞
3:     v.π = NIL
4: s.d = 0
```

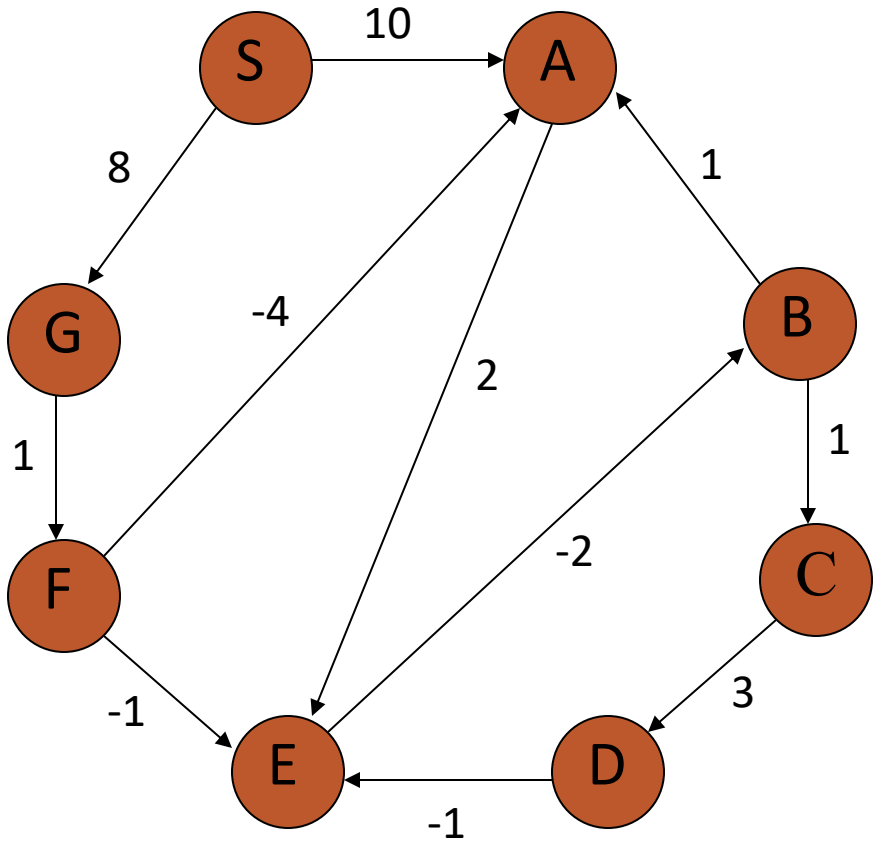
Algorithm 1 Bellman-Ford(G,s)

```
1: Init(G,s)
2: for n-1 times do
3:     for each edge (u, v) ∈ E do
4:         Relax(u, v)
```

Algorithm 2 RELAX

```
1: if v.d > u.d + w(u, v) then
2:     v.d = u.d + w(u, v)
3:     v.π = u
```

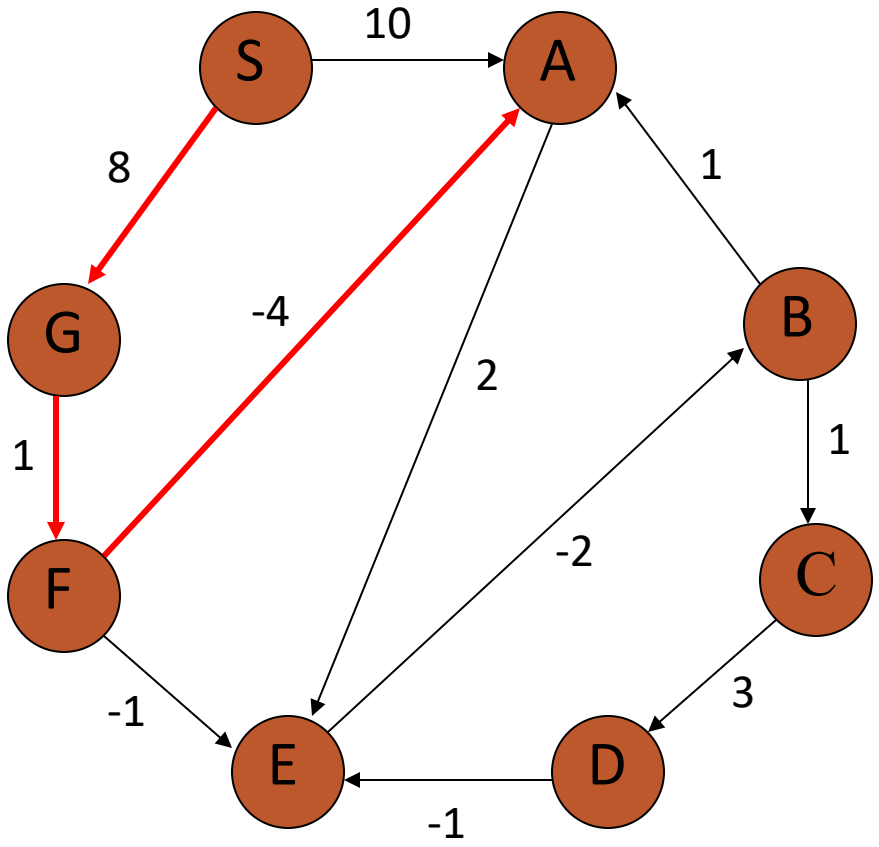
Bellman Ford



How many edges in the shortest path from s to:

A:

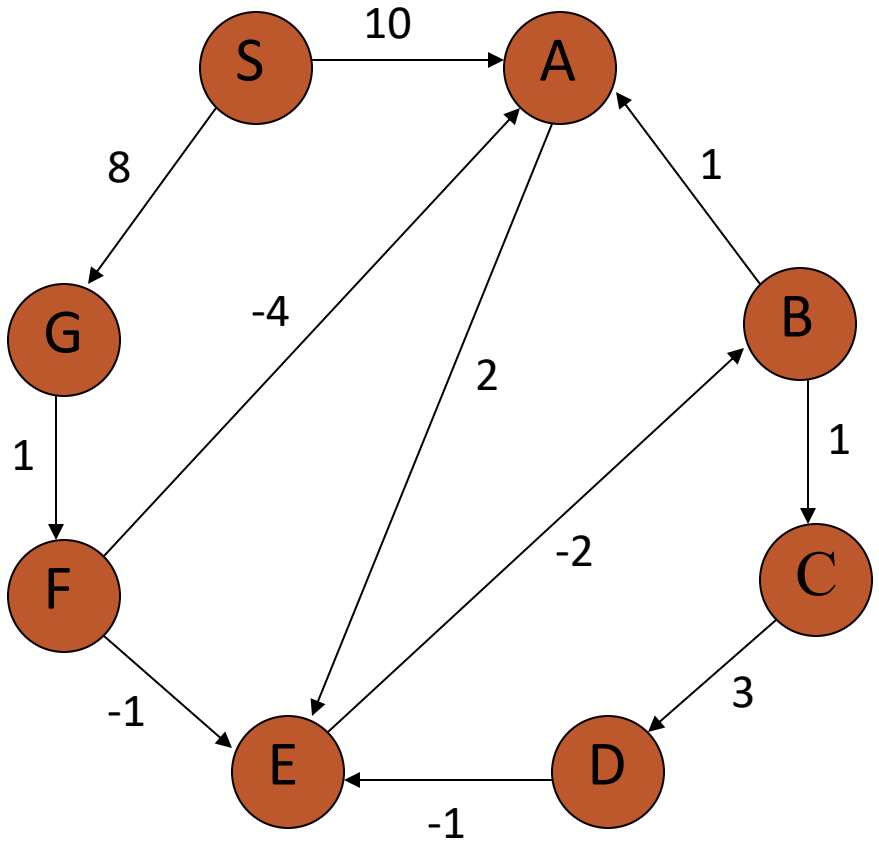
Bellman Ford



How many edges in the shortest path from s to:

A: 3

Bellman Ford

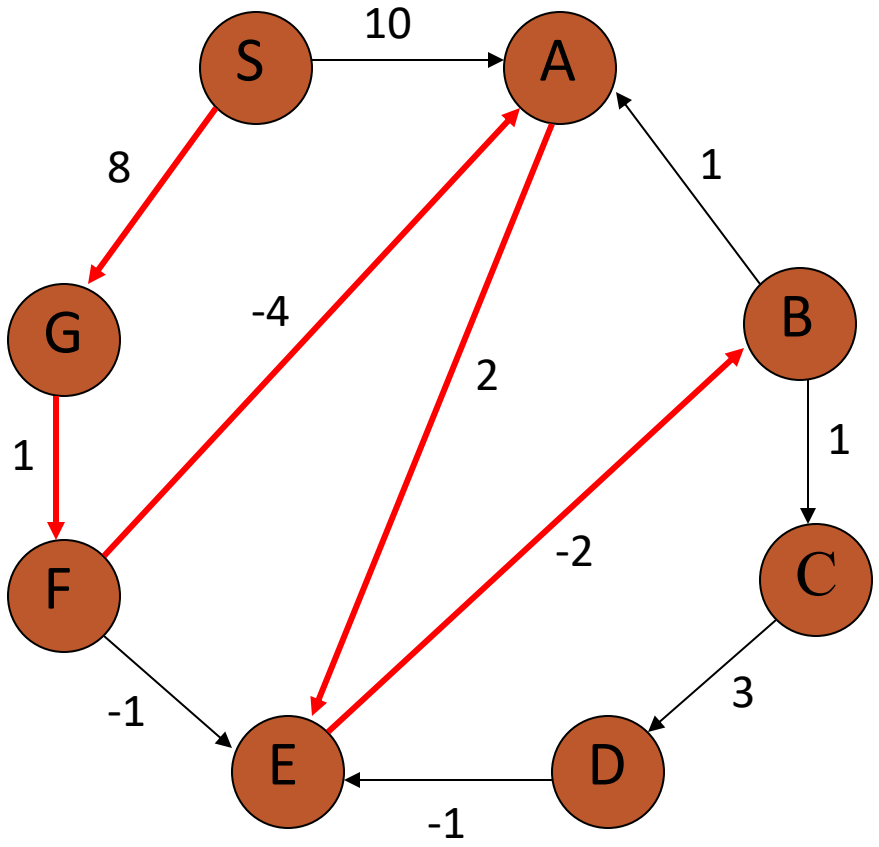


How many edges in the shortest path from s to:

A: 3

B:

Bellman Ford

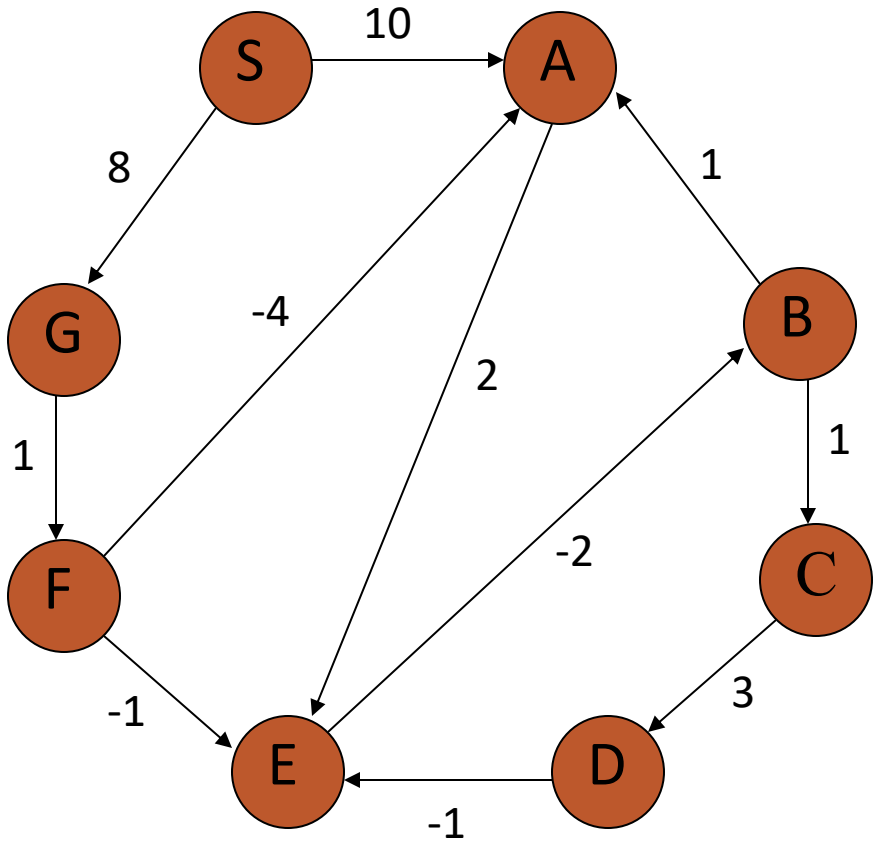


How many edges in the shortest path from s to:

A: 3

B: 5

Bellman Ford



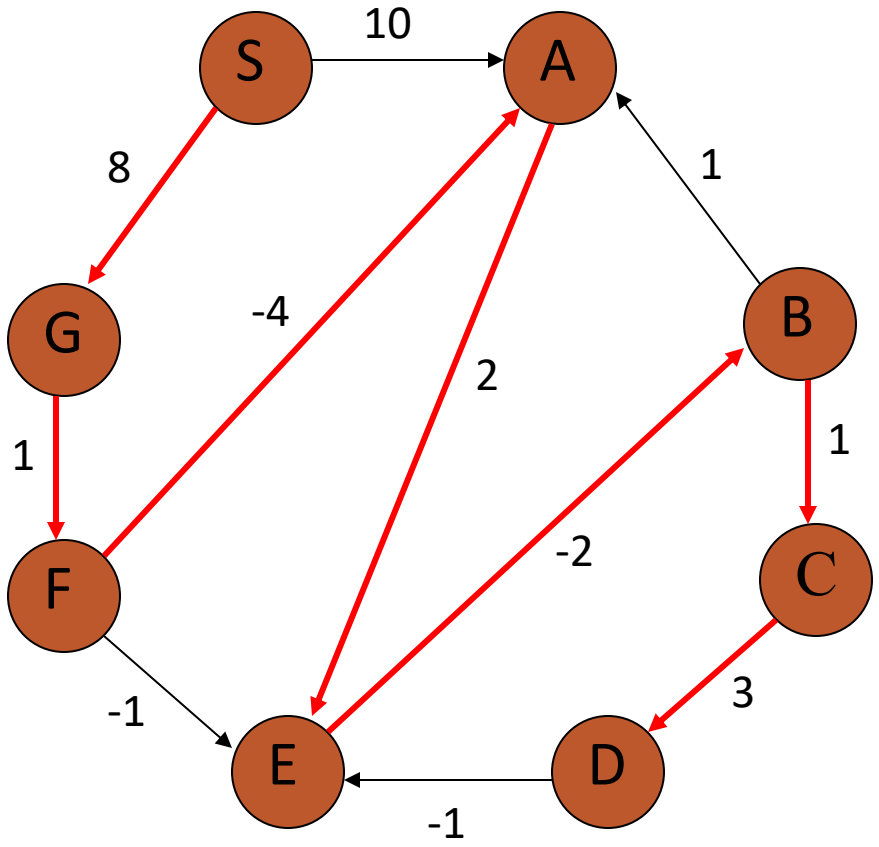
How many edges in the shortest path from s to:

A: 3

B: 5

D:

Bellman Ford



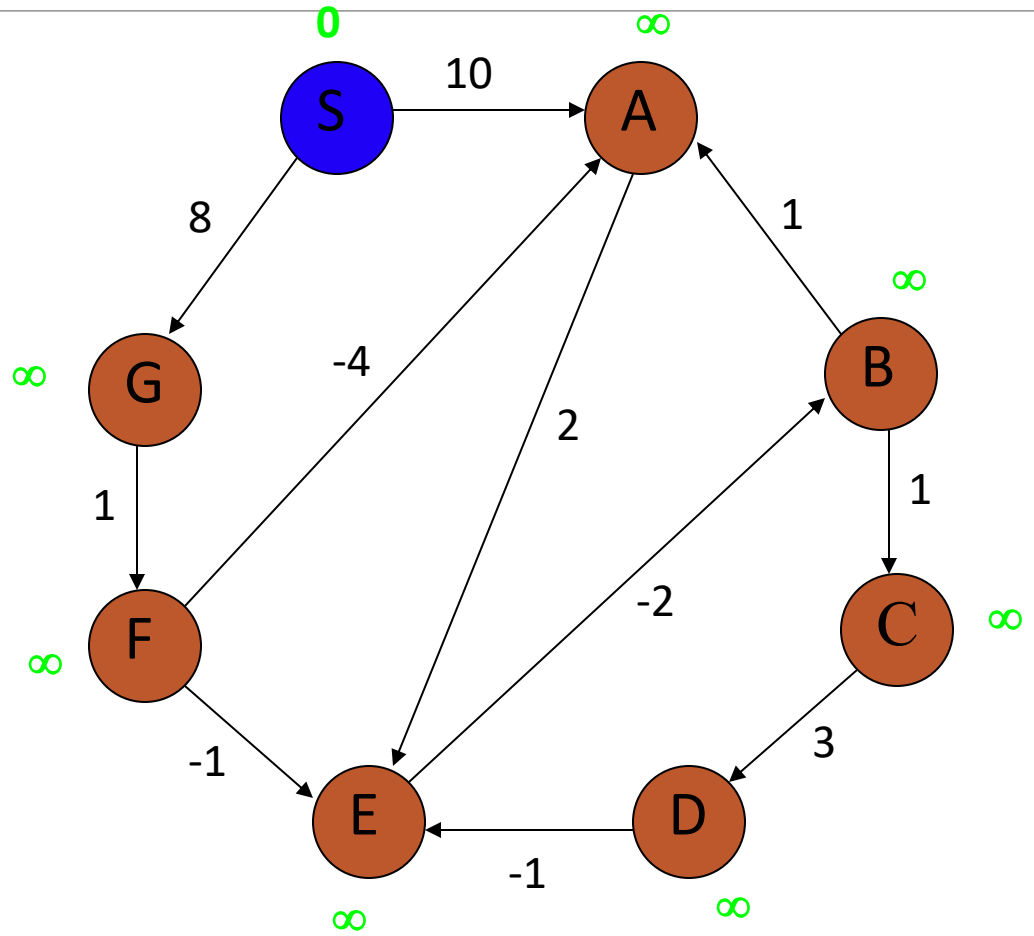
How many edges in the shortest path from s to:

A: 3

B: 5

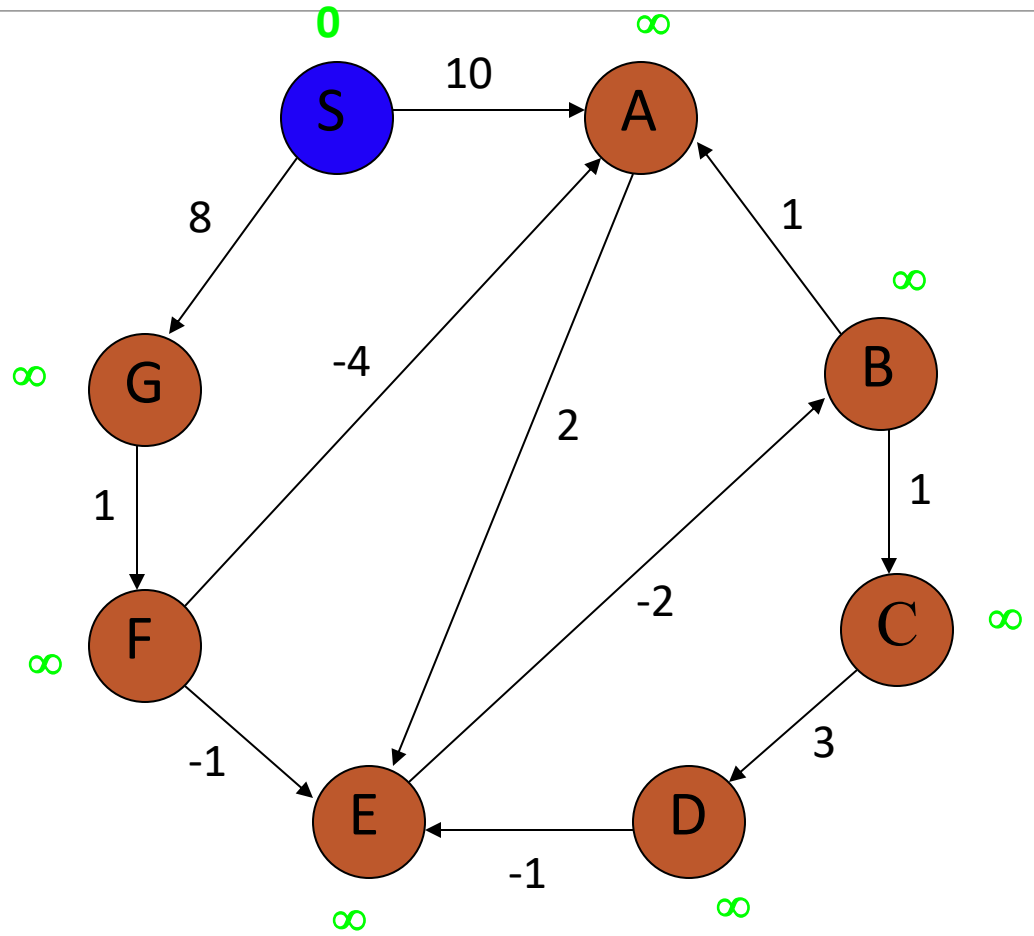
D: 7

Bellman Ford



Remark: If $v.d = u.d = \infty$, we do not update $v.d$, even if w(u,v)<0.

Bellman Ford

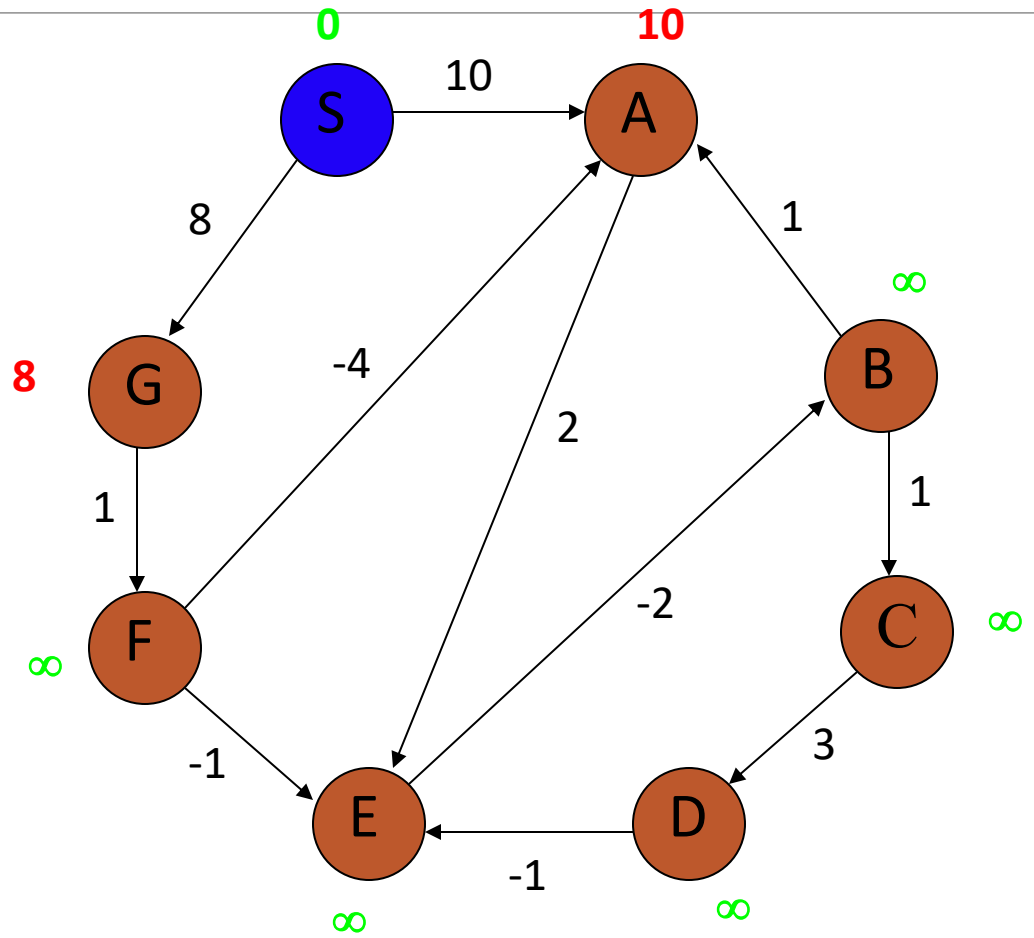


Initialization

Order of the edges:

(C,D), (B,C), (E,B), (D,E), (A,E), (F,E), (B,A), (F,A), (G,F), (S,A), (S,G)

Bellman Ford

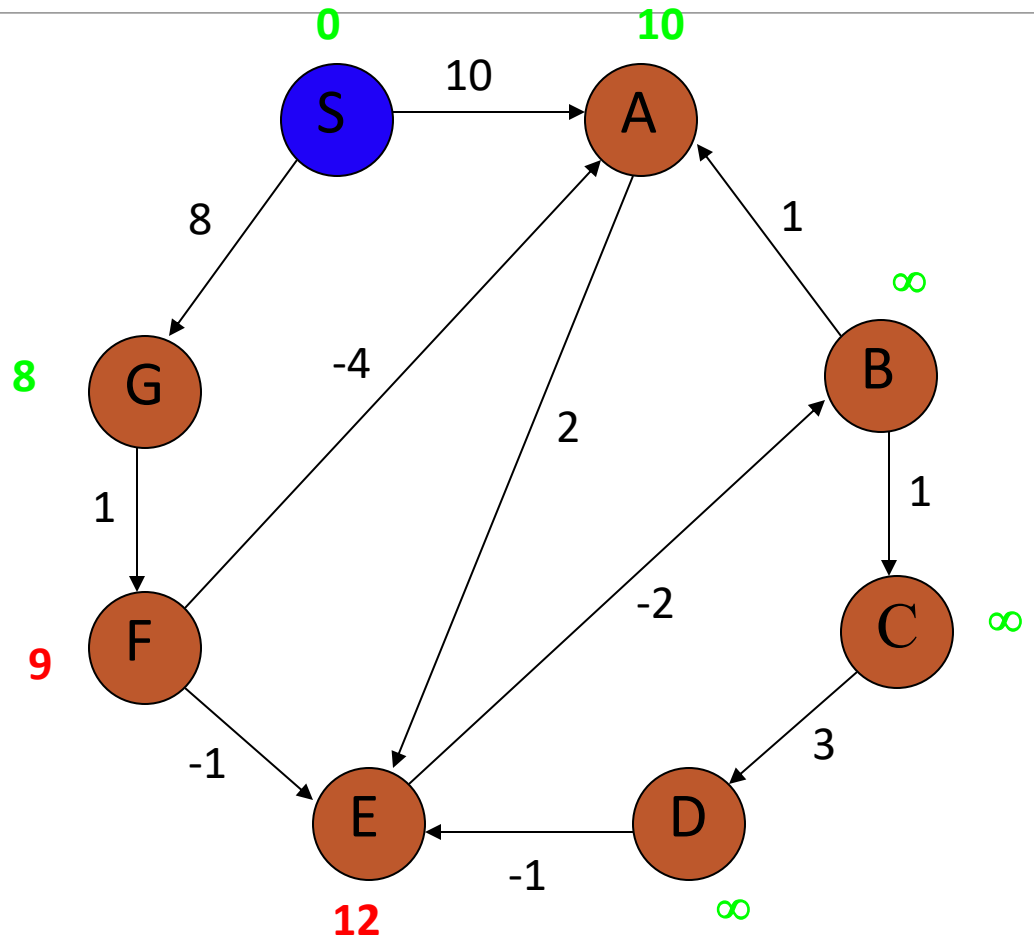


Sweep: 1

Order of the edges:

(C,D), (B,C), (E,B), (D,E), (A,E), (F,E), (B,A), (F,A), (G,F), (S,A), (S,G)

Bellman Ford

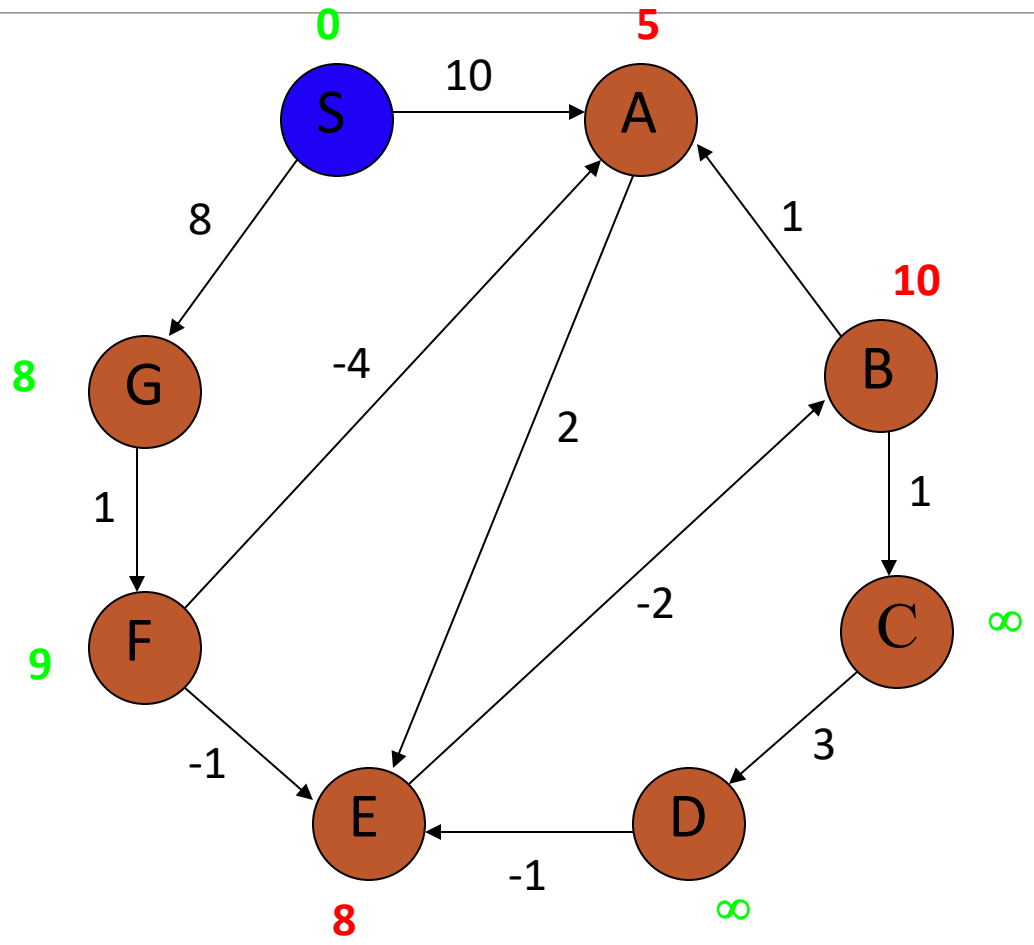


Sweep: 2

Order of the edges:

(C,D), (B,C), (E,B), (D,E), (A,E), (F,E), (B,A), (F,A), (G,F), (S,A), (S,G)

Bellman Ford



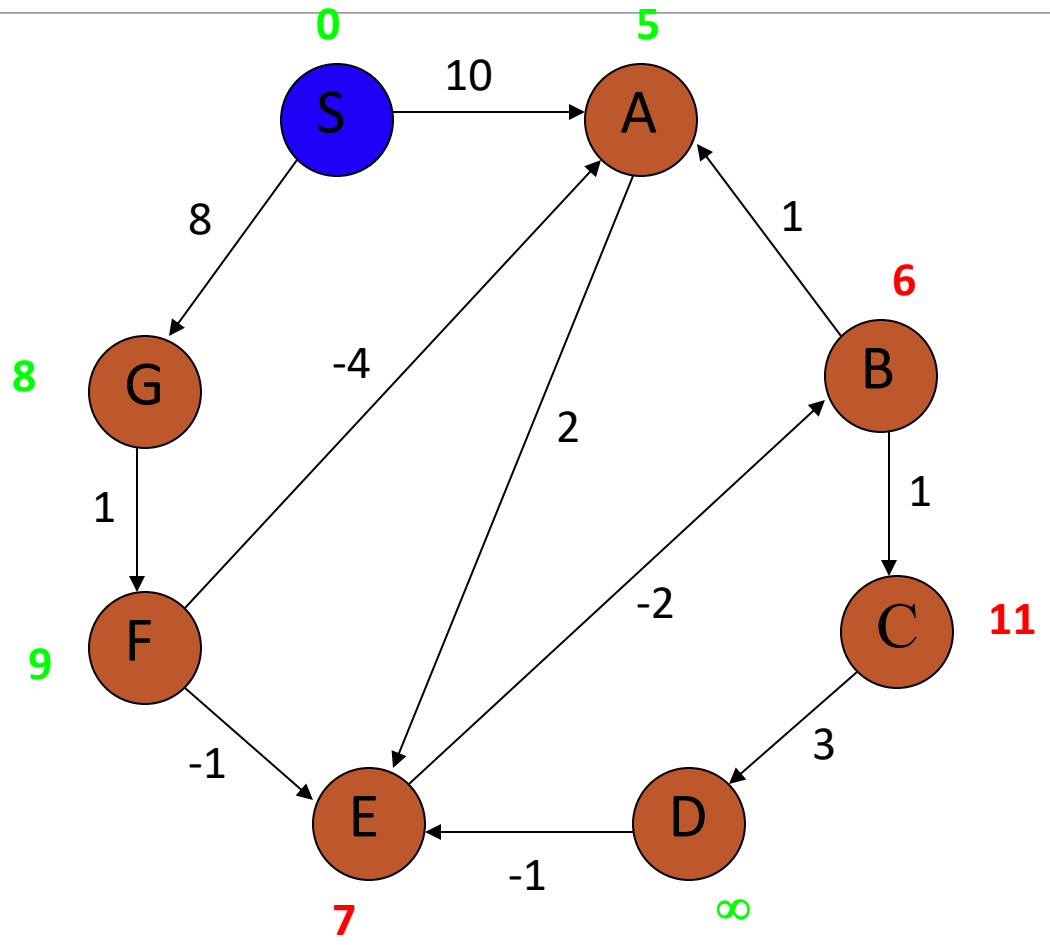
Sweep: 3

A has the correct distance and path

Order of the edges:

(C,D), (B,C), (E,B), (D,E), (A,E), (F,E), (B,A), (F,A), (G,F), (S,A), (S,G)

Bellman Ford

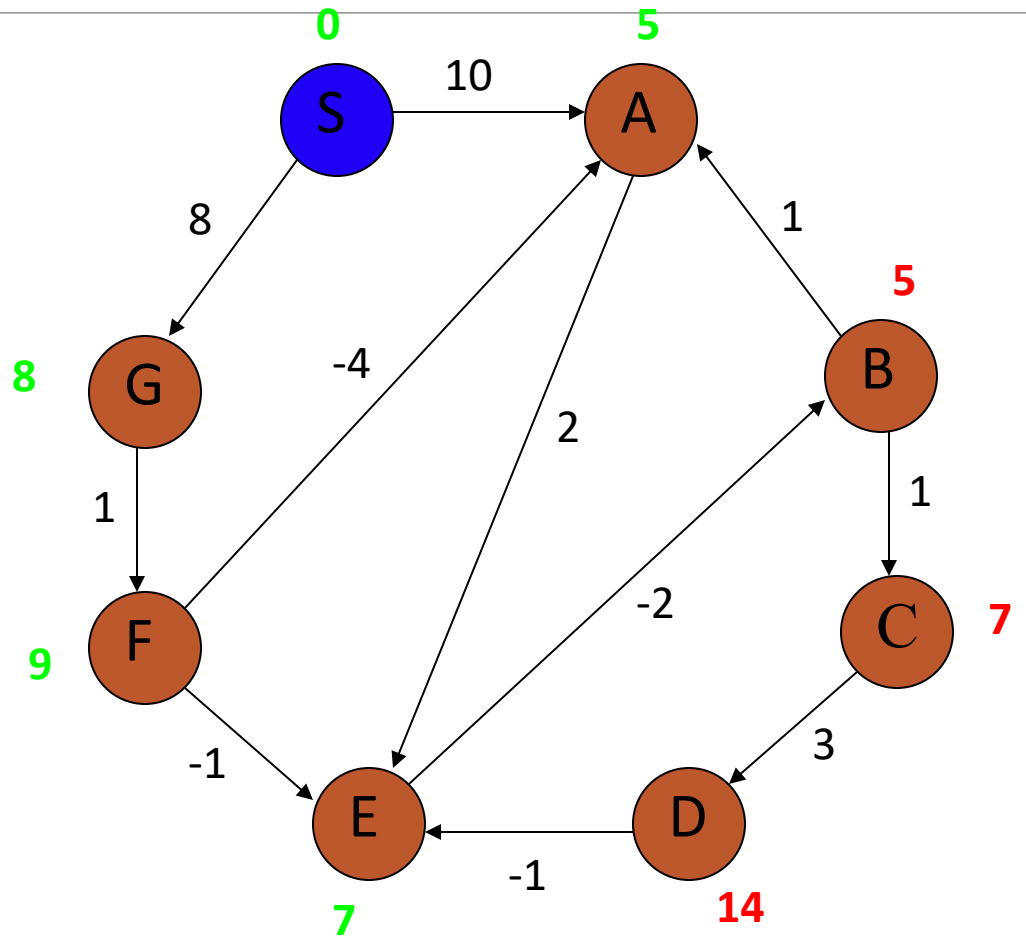


Sweep: 4

Order of the edges:

(C,D), (B,C), (E,B), (D,E), (A,E), (F,E), (B,A), (F,A), (G,F), (S,A), (S,G)

Bellman Ford



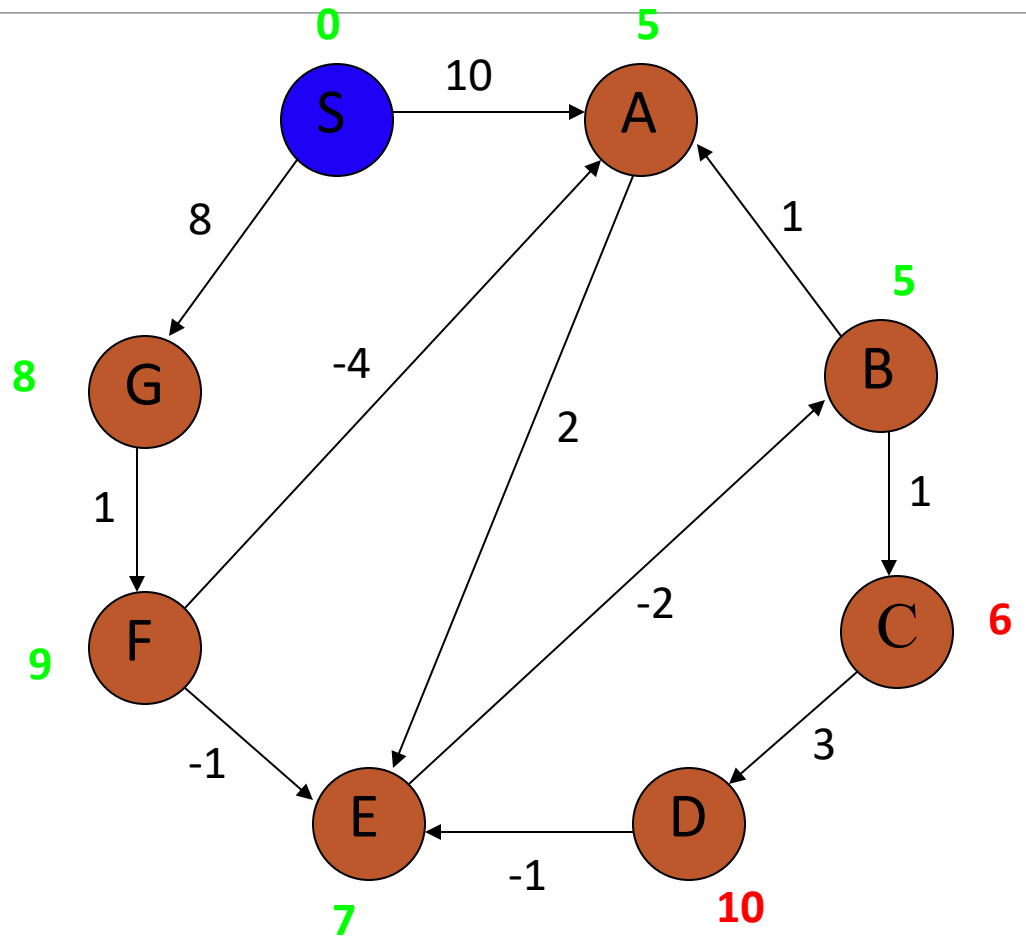
Sweep: 5

B has the correct distance and path

Order of the edges:

(C,D), (B,C), (E,B), (D,E), (A,E), (F,E), (B,A), (F,A), (G,F), (S,A), (S,G)

Bellman Ford

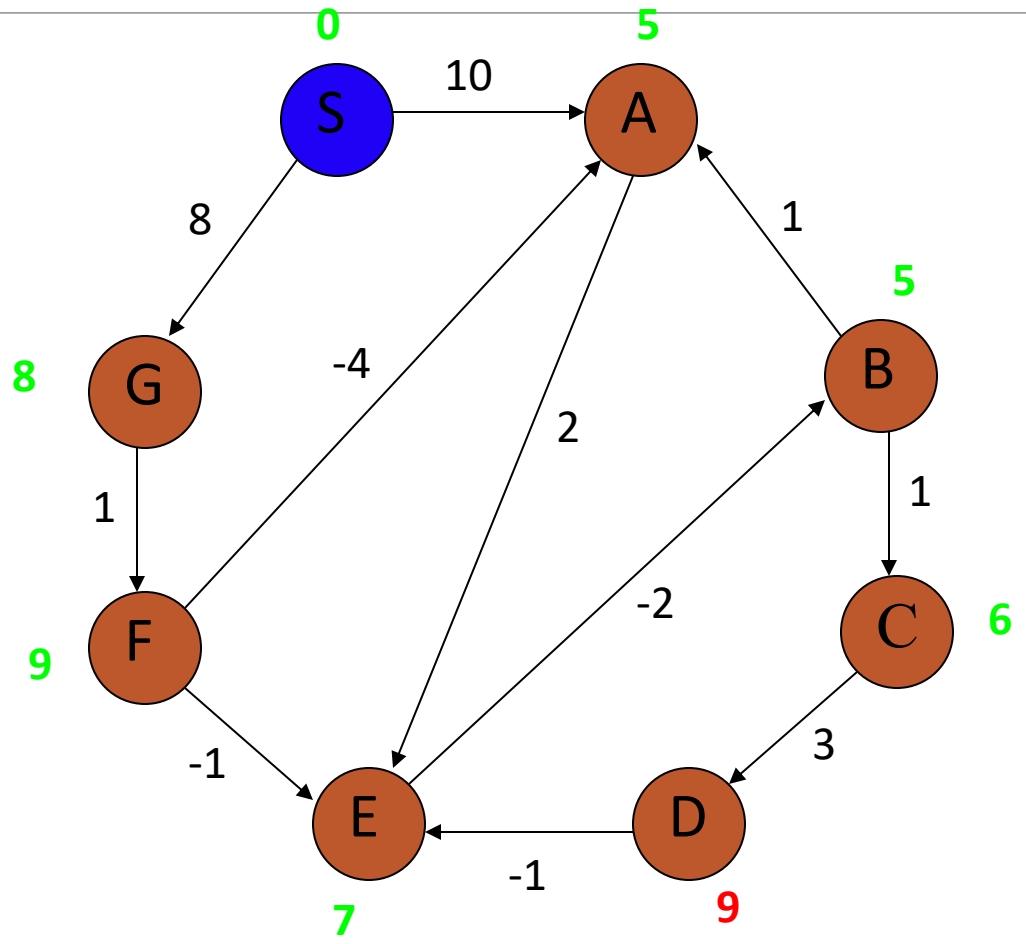


Sweep: 6

Order of the edges:

(C,D), (B,C), (E,B), (D,E), (A,E), (F,E), (B,A), (F,A), (G,F), (S,A), (S,G)

Bellman Ford



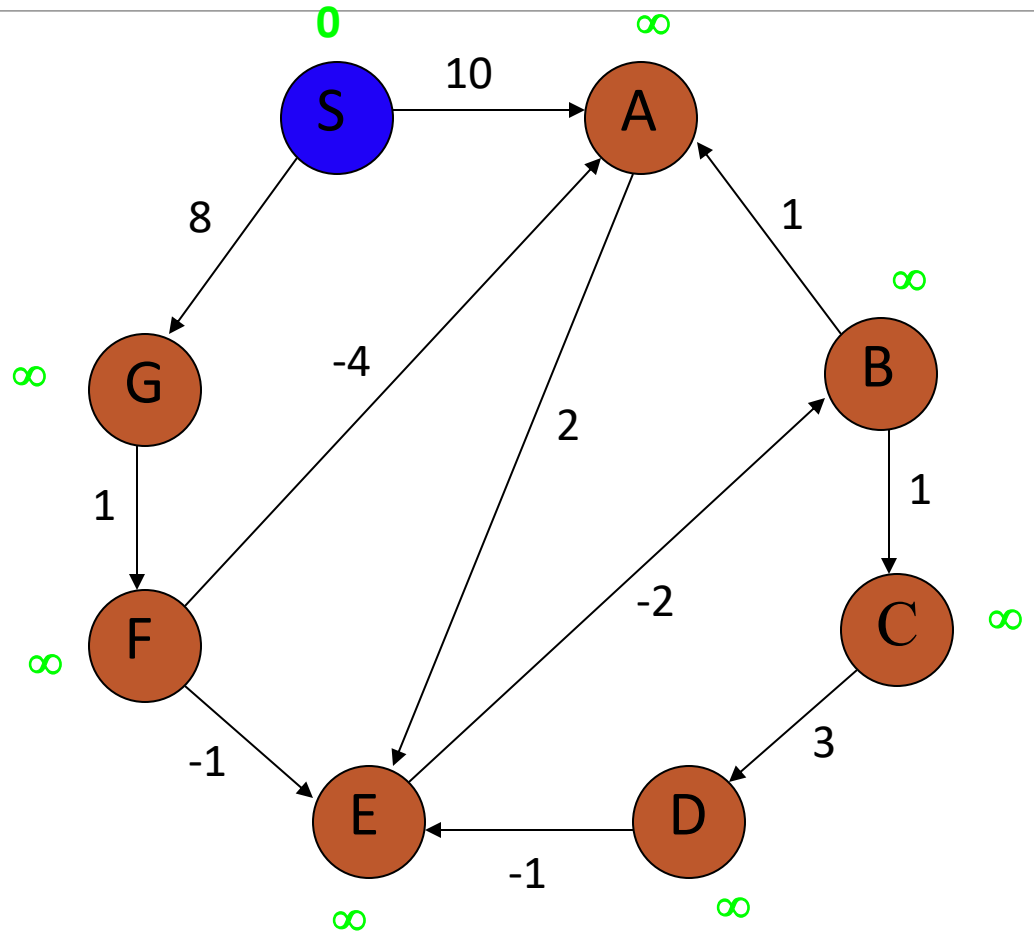
Sweep: 7

D (and all other nodes) have the correct distance and path

Order of the edges:

(C,D), (B,C), (E,B), (D,E), (A,E), (F,E), (B,A), (F,A), (G,F), (S,A), (S,G)

Bellman Ford



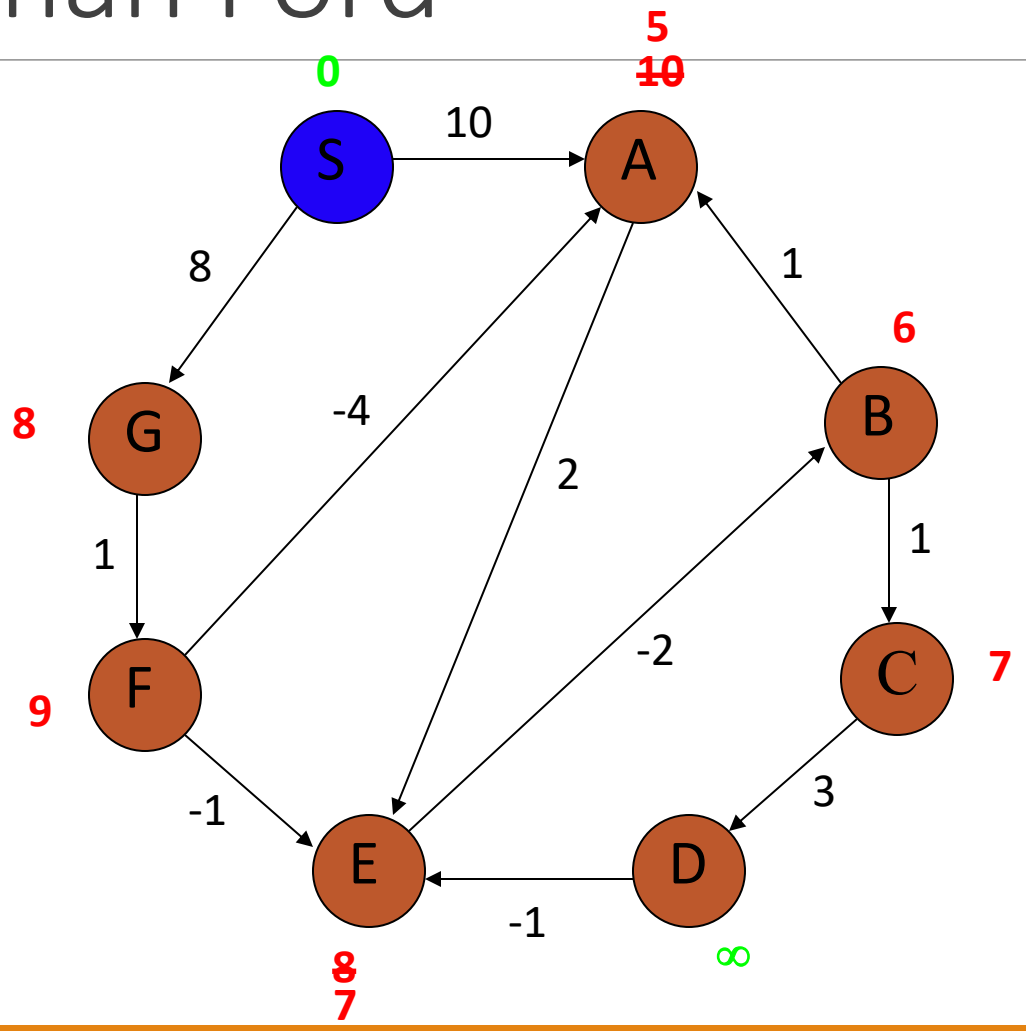
Initialization

Order of the edges:

(S,A), (S,G), (G,F), (F,E), (F,A), (C,D)

(E,B), (B,C), (B,A), (A,E), (D,E)

Bellman Ford



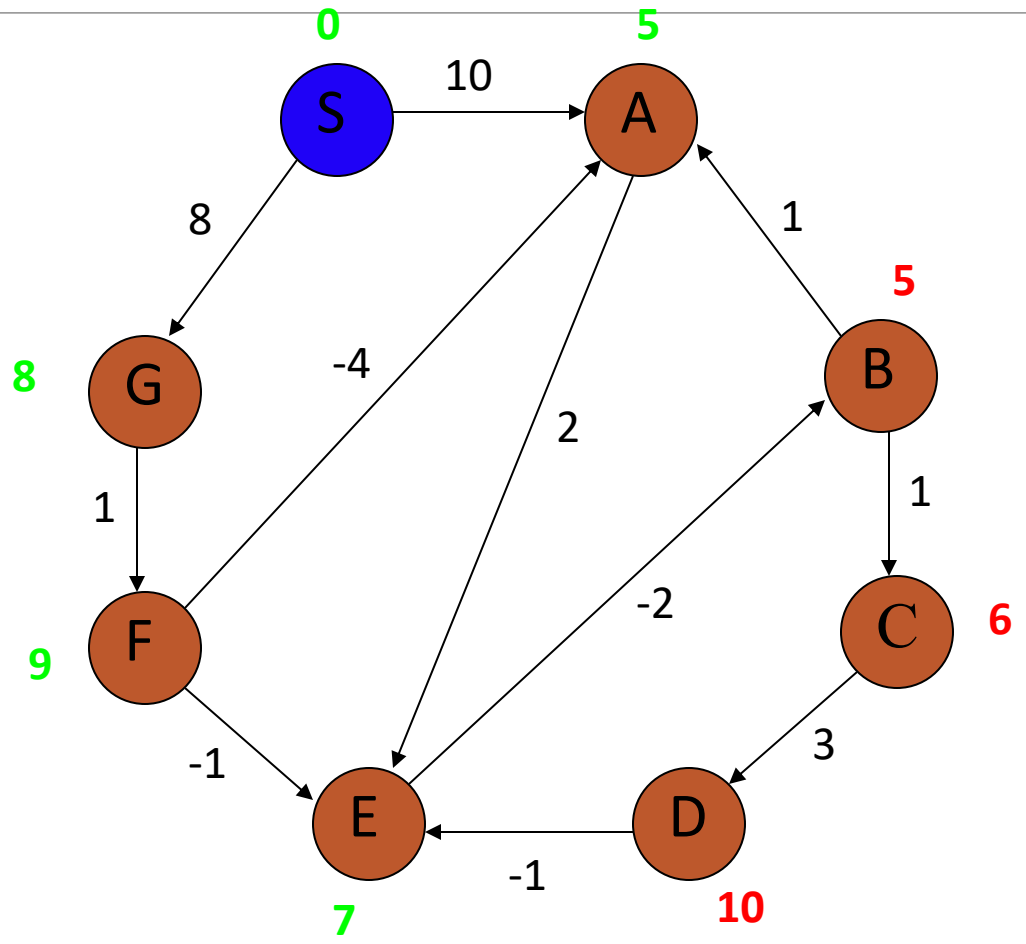
Sweep: 1

Order of the edges:

(S,A), (S,G), (G,F), (F,E), (F,A), (C,D)

(E,B), (B,C), (B,A), (A,E), (D,E)

Bellman Ford



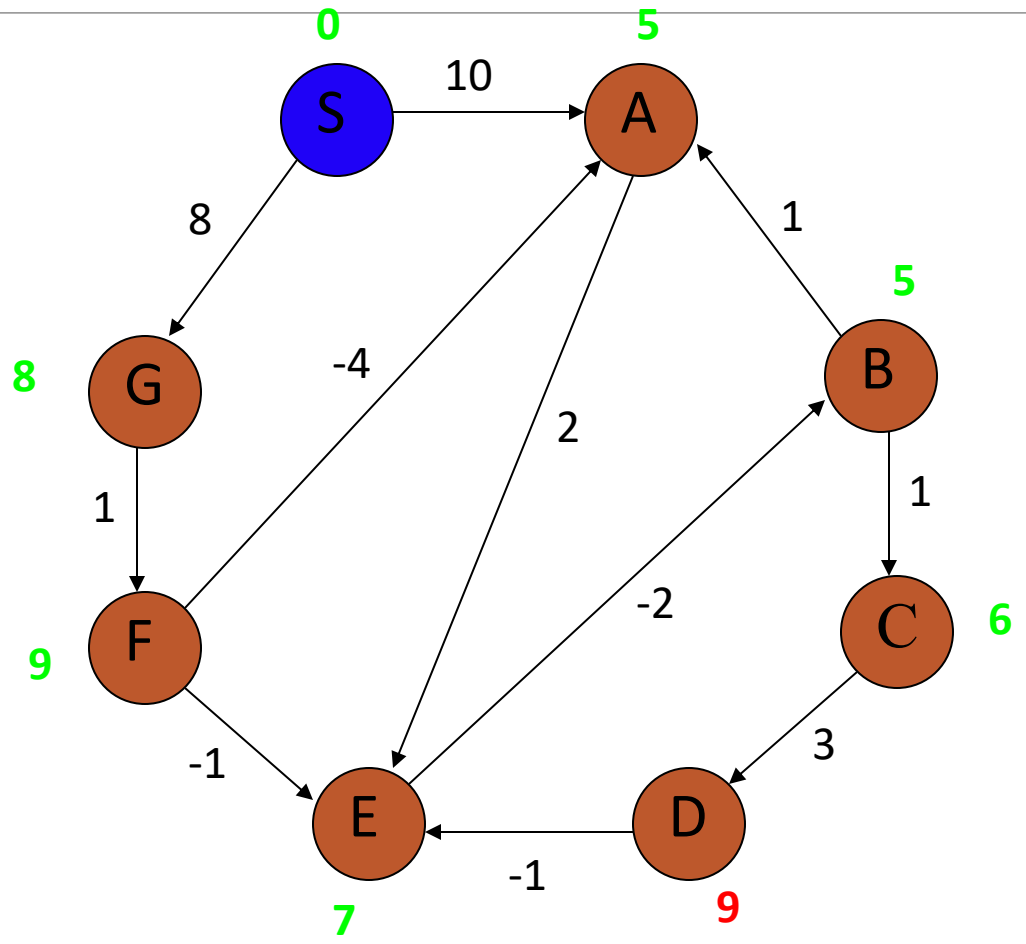
Sweep: 2

Order of the edges:

(S,A), (S,G), (G,F), (F,E), (F,A), (C,D)

(E,B), (B,C), (B,A), (A,E), (D,E)

Bellman Ford



Sweep: 3

Order of the edges:

(S,A), (S,G), (G,F), (F,E), (F,A), (C,D)

(E,B), (B,C), (B,A), (A,E), (D,E)

Exercise 1

For every $n \geq 1$:

1. Find a graph with n vertices where BF is done after a single iteration, regardless of the ordering of the edges.

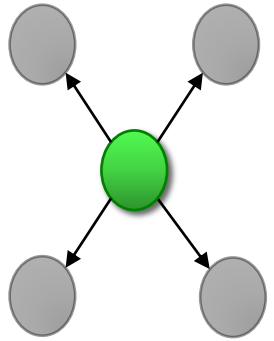
Exercise 1 - Solution

For every $n \geq 1$:

1. Find a graph with n vertices where BF is done after a single iteration, regardless of the ordering of the edges.

We want that all the shortest paths will consist of a single edge. This happens when all the vertices are adjacent to the source.

For example, in a directed star, in which $E = \{(s, v) | s \neq v\}$.



Exercise 1

For every $n \geq 1$:

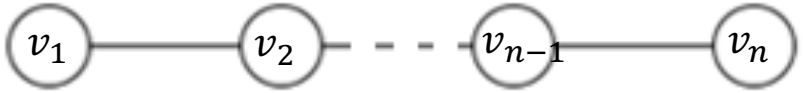
2. Find a graph with n vertices where n-1 iterations are essential in some run of BF.

Exercise 1 - Solution

For every $n \geq 1$:

2. Find a graph with n vertices where n-1 iterations are essential in some run of BF.

We want a path that contains n-1 edges. Take the path graph ("line"), with $s = v_1$ and the order of the edges $(v_{n-1}, v_n), \ldots, (v_1, v_2)$.



Floyd Warshall

- Shortest path between every pair of vertices
- Supports graphs with arbitrary weights

Algorithm 2 Floyd-Warshall(G,s)

1: Set $\delta^0(i,j) = \begin{cases} 0 & i = j \\ w(i,j) & (i,j) \in E \\ \infty & o.w. \end{cases}$

2: **for** For $k = 1$ to n **do**

3: **for** i = 1 to n **do**

4: **for** j = 1 to n **do**

5: $\delta^k(i,j) = \min\{\delta^{k-1}(i,j),\ \delta^{k-1}(i,k) + \delta^{k-1}(k,j)\}$

Floyd Warshall

- Lemma: If there are no negative cycles, $\delta^n(i,j) = d(i,j)$.
- Lemma: There exists a negative cycle iff there exists $1 \leq i \leq n$ such that $\delta^n(i,i) < 0$.

Exercise 2

G is a directed graph with arbitrary weights on the edges (but no negative cycles), and n>10 vertices. During an execution of Floyd-Warshall algorithm, there exists a pair of vertices i,j, for which the value of $\delta(i,j)$ decreases for the first time (from ∞ to some finite value) when $\delta^6(i,j)$ is calculated. Also, $\delta^n(i,j)= \delta^6(i,j)$.

1. True/False: every shortest path from i to j passes only in {1,2,3,4,5,6} as internal nodes.

Exercise 2 - Solution

G is a directed graph with arbitrary weights on the edges (but no negative cycles), and n>10 vertices. During an execution of Floyd-Warshall algorithm, there exists a pair of vertices i,j, for which the value of $\delta(i,j)$ decreases for the first time (from ∞ to some finite value) when $\delta^6(i,j)$ is calculated. Also, $\delta^n(i,j)= \delta^6(i,j)$.

1. True/False: every shortest path from i to j passes only in {1,2,3,4,5,6} as internal nodes.

False – it is possible for example that i-7-j is another SP.

Exercise 2

G is a directed graph with arbitrary weights on the edges (but no negative cycles), and n>10 vertices. During an execution of Floyd-Warshall algorithm, there exists a pair of vertices i,j, for which the value of $\delta(i,j)$ decreases for the first time (from ∞ to some finite value) when $\delta^6(i,j)$ is calculated. Also, $\delta^n(i,j)= \delta^6(i,j)$.

2. True/False: There are at least 6 different shortest paths between i and j.

Exercise 2 - Solution

G is a directed graph with arbitrary weights on the edges (but no negative cycles), and n>10 vertices. During an execution of Floyd-Warshall algorithm, there exists a pair of vertices i,j, for which the value of $\delta(i,j)$ decreases for the first time (from ∞ to some finite value) when $\delta^6(i,j)$ is calculated. Also, $\delta^n(i,j)= \delta^6(i,j)$.

2. True/False: There are at least 6 different shortest paths between i and j.

False – if for example there are only two edges i-6 and 6-j in the graph.

Exercise 2

G is a directed graph with arbitrary weights on the edges (but no negative cycles), and n>10 vertices. During an execution of Floyd-Warshall algorithm, there exists a pair of vertices i,j, for which the value of $\delta(i,j)$ decreases for the first time (from ∞ to some finite value) when $\delta^6(i,j)$ is calculated. Also, $\delta^n(i,j)= \delta^6(i,j)$.

3. True/False: there exists a shortest path from i to j that passes only in {1,2,3,4,5,6} as internal nodes.

Exercise 2 - Solution

G is a directed graph with arbitrary weights on the edges (but no negative cycles), and n>10 vertices. During an execution of Floyd-Warshall algorithm, there exists a pair of vertices i,j, for which the value of $\delta(i,j)$ decreases for the first time (from ∞ to some finite value) when $\delta^6(i,j)$ is calculated. Also, $\delta^n(i,j)= \delta^6(i,j)$.

3. True/False: there exists a shortest path from i to j that passes only in {1,2,3,4,5,6} as internal nodes.

True – because $\delta^n(i,j)= \delta^6(i,j)$.

Exercise 2

G is a directed graph with arbitrary weights on the edges (but no negative cycles), and n>10 vertices. During an execution of Floyd-Warshall algorithm, there exists a pair of vertices i,j, for which the value of $\delta(i,j)$ decreases for the first time (from ∞ to some finite value) when $\delta^6(i,j)$ is calculated. Also, $\delta^n(i,j)= \delta^6(i,j)$.

4. True/False: there is no path from i to j that passes only in {1,2,3,4,5} as internal nodes.

Exercise 2 - Solution

G is a directed graph with arbitrary weights on the edges (but no negative cycles), and n>10 vertices. During an execution of Floyd-Warshall algorithm, there exists a pair of vertices i,j, for which the value of $\delta(i,j)$ decreases for the first time (from ∞ to some finite value) when $\delta^6(i,j)$ is calculated. Also, $\delta^n(i,j)= \delta^6(i,j)$.

4. True/False: there is no path from i to j that passes only in {1,2,3,4,5} as internal nodes.

True – as otherwise the value of $\delta(i,j)$ would be finite before $\delta^6(i,j)$ is calculated.

Exercise 3

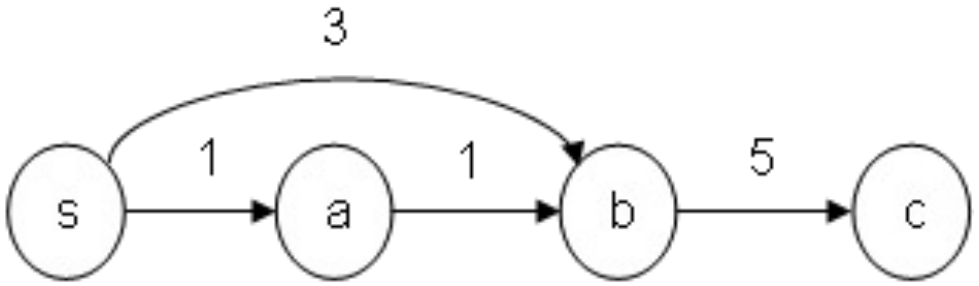
- Let G=(V,E) be a directed graph with non-negative costs on the edges. Let $s \in V$ be a source vertex.
- According to a new tax rule, the cost of the last edge in any path consisting of an odd number of edges is doubled.
- Suggest an efficient algorithm for computing, for every $v \in V$, the cheapest path from s to v, considering the above tax rule.
- Describe the algorithm, justify its correctness, and analyze its time complexity.

Exercise 3

For example: in the given graph,

d(s,s)=0, d(s,a)=2, d(s,b)=2, d(s,c)=8.

The cheapest path to nodes 'a' and 'c' is affected by the tax. The cost of the path s-a-b-c is 12.



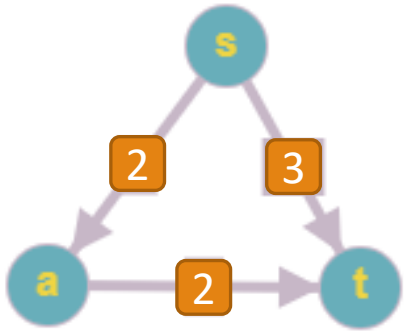
Remember: The tax is applied only on paths consisting of an odd number of edges.

Exercise 3 - Solution

Find a shortest path, then check whether it has an odd number of edges (backtrack to s using the parent field to check it), and if yes, add the cost of the last edge.

WRONG

For this graph and a path s-t:



- This algorithm will return 6.
- The correct answer is 4.
- See also node c in previous example
- We must also check a candidate path with an even number of edges

Exercise 3 - Solution

1. Construct the graph G'=(V', E').

V'= $V^0 \cup V^1 \cup V^2$, where each of V^0, V^1, V^2 is a copy of V.

E' = $\{(v^0, u^1), (v^1, u^0) \mid$ for each (v,u)$\in$E$\} \cup \{(v^0, u^2) \mid$ for each (v,u)$\in$E$\}$.

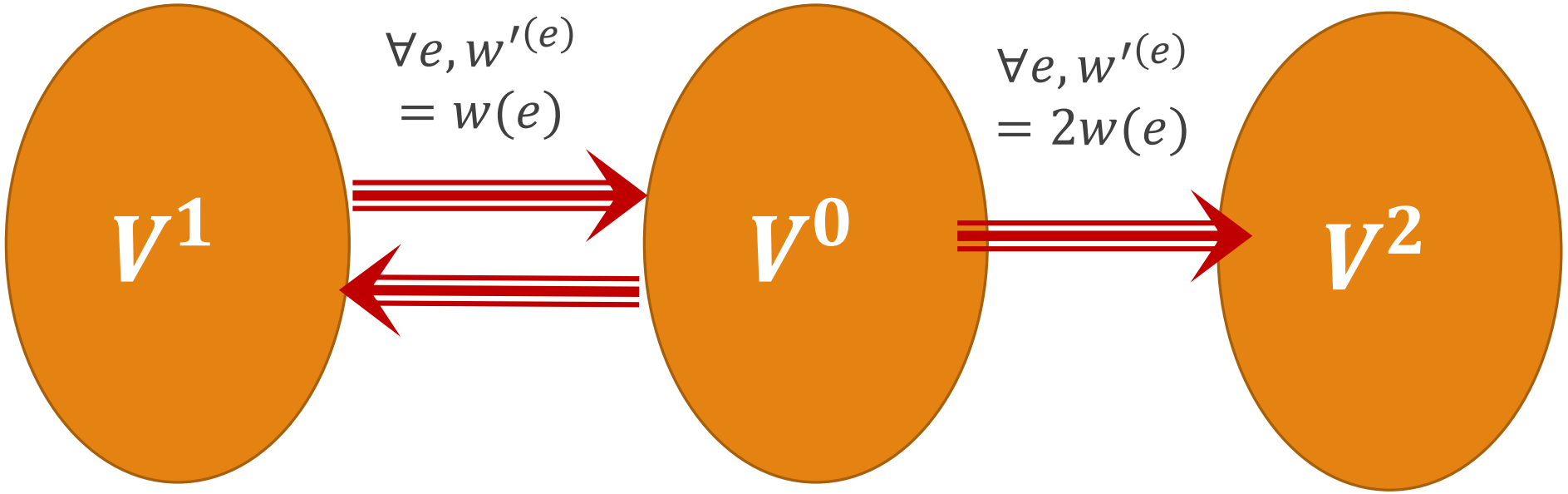
The weights of edges between the first two copies (edges of type (v^0, u^1) and (v^1, u^0)) are as in the original graph. The weight of edges from the first to the third copy (edges of type (v^0, u^2)) is 2·the original weights.

2. Run Dijksrta algorithm on G' from vertex s^0 as a source.

3. Return, for each vertex v, the minimum of $\{d(s^0, v^0), d(s^0, v^2)\}$.

Exercise 3 - Solution

Illustration of the graph G'=(V', E')



Exercise 3 - Solution

Correctness:

Main Claim:

1. There is a path p' in G' from s^0 to v^0 and cost c, *if and only if* there is a path p in G from s to v, having an even number of edges and cost c.

2. There is a path p' in G' from s^0 to v^2 having cost c *if and only if* there is a path p in G from s to v, having an odd number of edges and taxed-cost c.

Exercise 3 - Solution

Proof of 2 (1 is simpler)

Let p' be a path in G' from s^0 to v^2. By the construction, there are no internal edges in V^2, thus, edges from V^0 to V^2 can only be used as a last edge in a path, and only after a prefix of an even number of edges. Therefore p' has an odd number of edges.

By the construction, the edges of p' correspond to edges of a path p from s to v in G passing in the same vertices.

The last edge of p' corresponds to the last edge in p and its cost in G' is doubled, therefore the cost of p' is the taxed-cost of p.

Exercise 3 - Solution

Correctness (cont'):

Therefore, d(s^0, v^0) is the shortest path from s to v consisting of an even number of edges, while d(s^0, v^2) is the shortest path from s to v using an odd number of edges – with double-cost last edge.

Exercise 3 - Solution

Time complexity:

1. Building G': $|V'| = 3|V|$, $|E'| = 3|E|$. Therefore, the size of G' is $O(|V|+|E|)$
2. Running Dijkstra on a graph with 3n vertices and 3m edges: $O(m+n\log n)$ [it is OK to write $O((n+m)\log n)$ or $O(n^2)$]
3. Calculating $\min\{d(s^0, v^0), d(s^0, v^2)\}$ and output take O(n) additional time.

All together: O(m+nlogn).

Exercise 4

True or False:

1. Let G be an undirected graph. If there exists a single BFS tree rooted at s, then G contains no cycles.

Exercise 4 – Solution (1)

True or False:

1. Let G be an undirected graph. If there exists a single BFS tree rooted at s, then G contains no cycles.

False. For example, a simple cycle of length 3.

Exercise 4

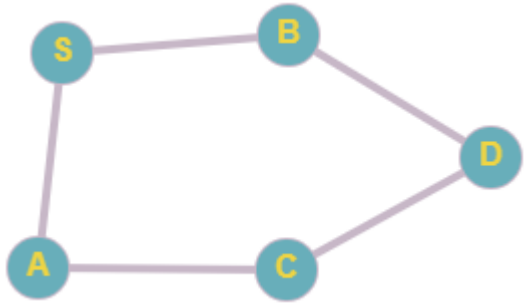
True or False:

2. Let G be an undirected graph. We perform DFS(s) followed by BFS(s), where the same data structure is used in both runs. Let u be some vertex in G. If u is a leaf in the DFS tree then it must be a leaf in the BFS tree.

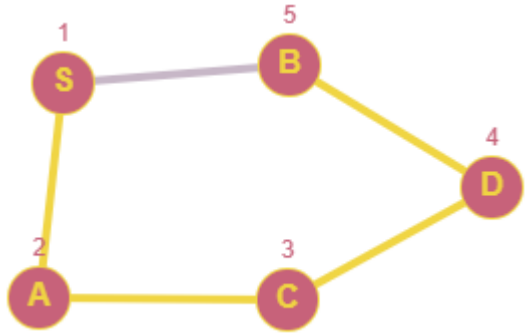
Exercise 4 – Solution (2)

False. In this graph, B is a leaf in DFS(s), but not a leaf in BFS(s).

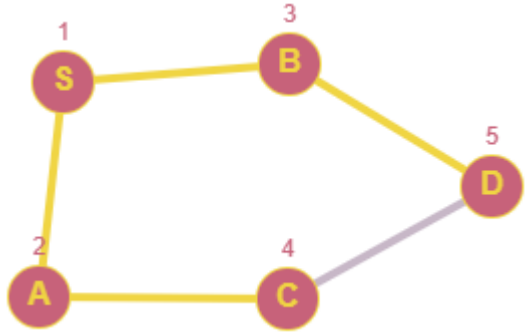
G



DFS(s)



BFS(s)



Exercise 5

Arbitrage is the use of discrepancies in currency exchange rates to transform one unit of a currency into more than one unit of the same currency. For example, consider the following relations:

Original currency	New currency
1 USD	0.7 British pounds
1 British pound	9.5 Franc
1 Franc	0.16 USD

If we start with 1 USD, we can gain 0.064! (0.7*9.5*0.16=1.064)

Exercise 5

Suppose we are given n currencies and $n \times n$ table of exchange rates such that one unit of currency c_i buys A[i,j]>0 units of currency c_j.

Suggest an efficient algorithm to determine whether or not there exists a sequence of currencies such that:

$$A[i_1, i_2] \cdot A[i_2, i_3] \cdot \cdots \cdot A[i_k, i_1] > 1$$

Exercise 5 - Solution

How to represent this problem as a graph problem?

If we think of currency as vertices and exchange rates as weights on the edges, we can represent by a graph.

How to turn product into sum?

Applying log: $\log(A[i_1, i_2] \cdot A[i_2, i_3] \cdot \cdots \cdot A[i_k, i_1]) > \log(1)$ (the inequality holds since log is monotone).

$$\log(A[i_1, i_2]) + \cdots + \log(A[i_k, i_1]) > 0$$

If we think about log of rates as weights, all we need to find is a positive cycle…

Exercise 5 - Solution

We know how to find negative cycle – using Bellman-Ford.

Solution: algebraic manipulation on the input:

$$A[i_1, i_2] \cdot A[i_2, i_3] \cdot \cdots \cdot A[i_k, i_1] > 1 \Leftrightarrow$$

$$1/A[i_1,i_2] \cdot \ldots \cdot 1/A[i_k,i_1] < 1 \Leftrightarrow$$

$$\log\left(1/A[i_1,i_2] \cdot \ldots \cdot 1/A[i_k,i_1]\right) < \log(1) \Leftrightarrow$$

$$\log\left(1/A[i_1,i_2]\right) + \cdots + \log\left(1/A[i_k,i_1]\right) < 0$$

Exercise 5 - Solution

Construct a graph where each vertex is a currency and each edge has a weight of $\log\left({}^{1}/_{A[i,j]}\right)$

Use Bellman-Ford algorithm to find a negative cycle.

If exists – return True. Else, return False.

Time Complexity: as Bellman-Fords.